一、实验目的

1 Verilog Vivado Logisim

2

二、实验内容

1

2

三、实验要求

1 Vivado Logisim

2 Verilog / / / Vivado Logisim

3 Vivado /Logism

四、实验过程及分析

1 1

1

```
module fulladder(
    input a,
    input b,
    input cin,
    output sum,
    output cout
);
    assign {cout, sum} = a + b + cin;
endmodule
```

a b cin sum cout sum = a ^ b ^ cin

cout = (a & b) | ((a ^ b) & cin) assign cout sum cout

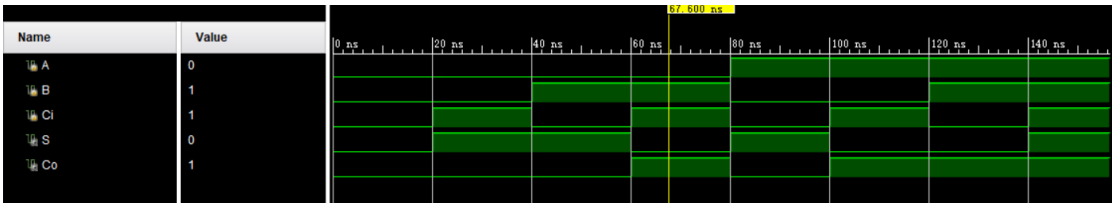
2

```
module sim_fulladder;
    reg A, B;
    reg Ci;
    wire S;
    wire Co;
    initial begin
        A = 1'b0; B = 1'b0; Ci = 0;
```

```
    end
    always begin
        #20 Ci = 1'b1;
        #20 B = B + 1'b1; Ci = ~Ci;
        #20 {A, B, Ci} = 3'b011;
        #20 {A, B, Ci} = 3'b100;
        repeat(3) begin
            #20 {A, B, Ci} = {A, B, Ci} + 1'b001;
        end
    end
    fulladder T(A, B, Ci, S, Co);
endmodule
```

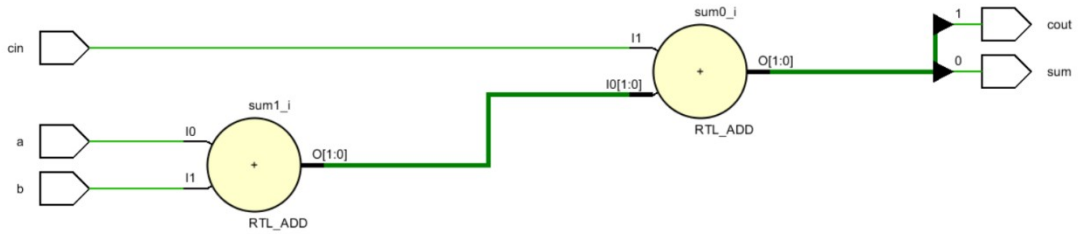
0 0+0+0 1+1+1

3 SIMULATION



sum cout (A,B,Ci)=(0,0,1) sum=1, cout=0 (1,1,1) sum=1, cout=1 (0,1,1) sum=0, cout=1

4 RTL ANALYSIS



2 4

1

```
module fulladder_4(
```

```
    input [3:0] a,
    input [3:0] b,
    input cin,
    output [3:0] sum,
    output cout
);
    wire [3:0] carry;
    fulladder fa0(a[0], b[0], cin, sum[0], carry[0]);
    fulladder fa1(a[1], b[1], carry[0], sum[1], carry[1]);
    fulladder fa2(a[2], b[2], carry[1], sum[2], carry[2]);
    fulladder fa3(a[3], b[3], carry[2], sum[3], cout);
endmodule
```

1

2

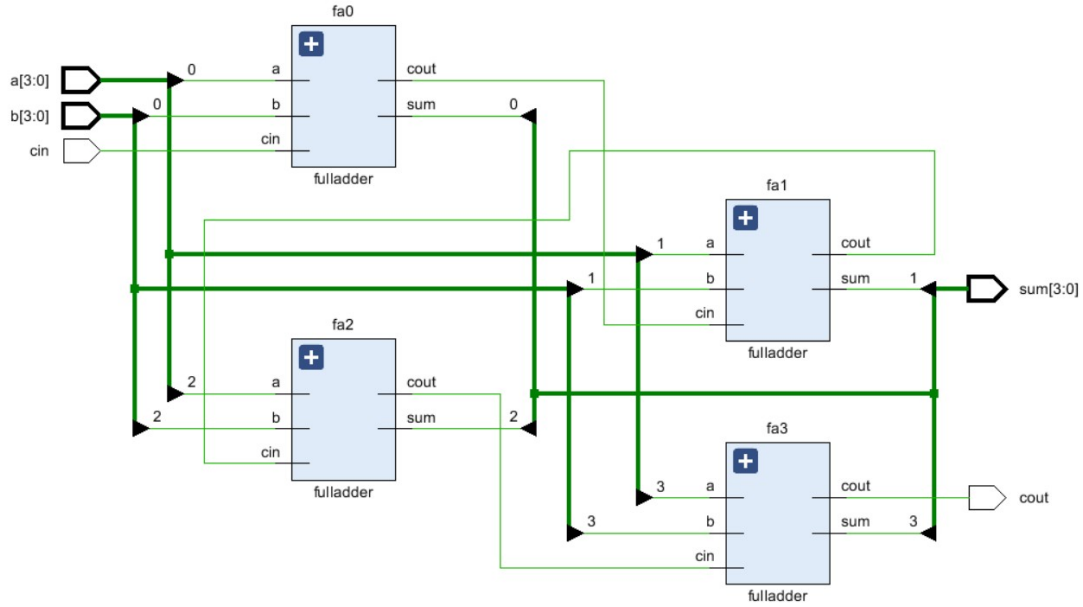
```
module sim_fulladder_4;
    reg [3:0] A, B;
    reg Ci;
    wire [3:0] S;
    wire Co;
    fulladder_4 T(A, B, Ci, S, Co);
    initial begin
        A = 4'b0000; B = 4'b0000; Ci = 1'b0;
    end
    always begin
        #20 Ci = 1'b1;
        #20 B = B + 1'b1; Ci = ~Ci;
        #20 {A, B, Ci} = 12'b0000_0001_1;
        #20 {A, B, Ci} = 12'b0001_0000_0;
        repeat(8) begin
            #20 {A, B, Ci} = {A, B, Ci} + 1'b1;
            #10 A    = A + 2'b11;
        end
    end
endmodule
```

repeat

3 SIMULATION

sum[3:0] cout A=0000, B=0000, Ci=1 S=0001, Cout=0 A=0111, B=0001, Ci=0 S=0100, Cout=0 A=4'b1101, B=4'b0010, Ci=1 Cout=1 sum=4'b0000 4

4 RTL ANALYSIS



3 16

1

```
module fulladder_16(
    input [15:0] a,
    input [15:0] b,
    input cin,
```

```
    output [15:0] sum,
    output cout
);
    wire [3:0] carry;

    fulladder_4 fa0(a[3:0],    b[3:0],    cin,          sum[3:0],    carry[0]);
    fulladder_4 fa1(a[7:4],    b[7:4],    carry[0],     sum[7:4],    carry[1]);
    fulladder_4 fa2(a[11:8],   b[11:8],   carry[1],     sum[11:8],   carry[2]);
    fulladder_4 fa3(a[15:12], b[15:12], carry[2],     sum[15:12], cout);
endmodule
```

16 4 4 4 fa0 a[3:0], b[3:0], cin sum[3:0], carry[0] 4 fa1 fa2 cin 4 fa3 cout

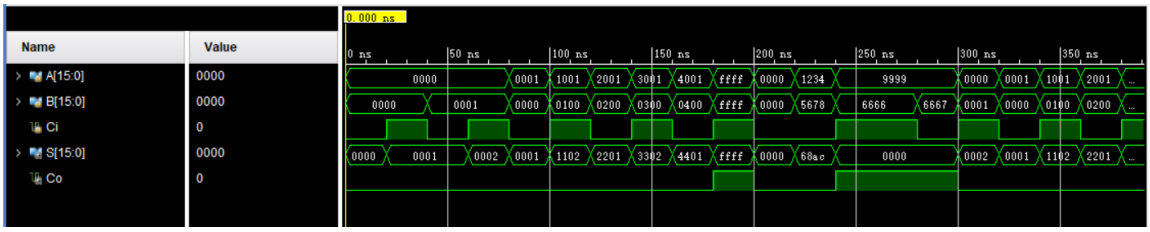
2

```
module sim_fulladder_16;
    reg [15:0] A, B;
    reg Ci;
    wire [15:0] S;
    wire Co;
    fulladder_16 T(A, B, Ci, S, Co);
    initial begin
        A = 16'h0000; B = 16'h0000; Ci = 1'b0;
    end
    always begin
        #20 Ci = 1'b1;
        #20 B = B + 16'h0001; Ci = ~Ci;
        #20 {A, B, Ci} = {16'h0000, 16'h0001, 1'b1};
        #20 {A, B, Ci} = {16'h0001, 16'h0000, 1'b0};
        repeat(4) begin
            #20 {A, B, Ci} = {A + 16'h1000, B + 16'h0100, ~Ci};
        end
        #20 {A, B, Ci} = {16'hFFFF, 16'hFFFF, 1'b1};
        #20 {A, B, Ci} = {16'h0000, 16'h0000, 1'b0};
        #20 {A, B, Ci} = {16'h1234, 16'h5678, 1'b0};
        #20 {A, B, Ci} = {16'h9999, 16'h6666, 1'b1};
    end
endmodule
```

A=16'h0000, B=16'h0000, Ci=0 A B Ci

1 (A=16'hFFFF,B=16'hFFFF,Ci=1) (0x1234,0x5678) (0x9999,0x6666)

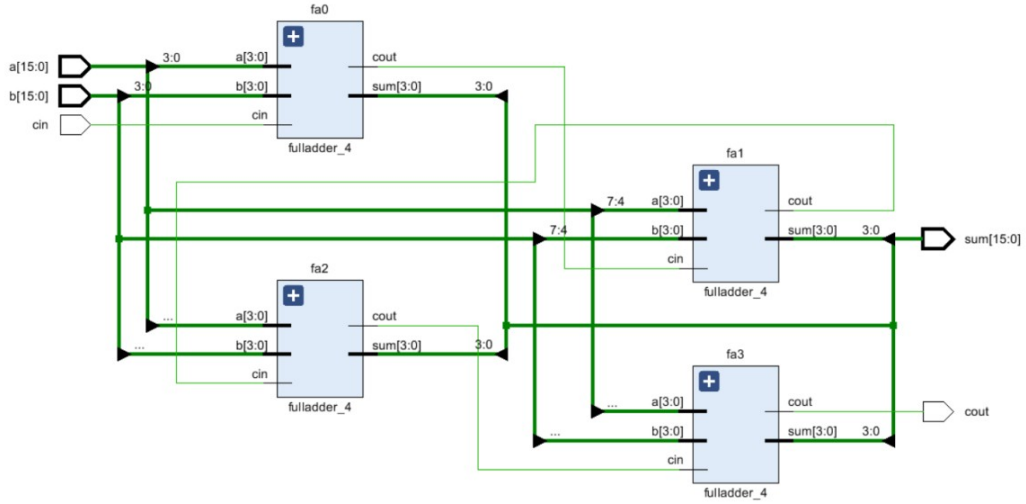
3 SIMULATION



sum[15:0] cout (A,B,Ci)=(0xFFFF,0xFFFF,1) 1FFFF 16

sum=0xFFFF cout=1 (1234,5678) 16

4 RTL ANALYSIS:



4 4

1

```
module CLA_4(
    input [3:0] A,
    input [3:0] B,
    input Cin,
    output [3:0] S,
    output Cout
);
    wire [3:0] G, P;
    wire [4:0] C;
```

```
    genvar i;
    generate
        for (i = 0; i < 4; i = i + 1) begin: bit_adder
            assign G[i] = A[i] & B[i];
            assign P[i] = A[i] | B[i];
            assign S[i] = A[i] ^ B[i] ^ C[i];
        end
    endgenerate
    assign C[0] = Cin;
    assign C[1] = G[0] | (P[0] & C[0]);
    assign C[2] = G[1] | (P[1] & G[0]) | (P[1] & P[0] & C[0]);
    assign C[3] = G[2] | (P[2] & G[1]) | (P[2] & P[1] & G[0]) |
                  (P[2] & P[1] & P[0] & C[0]);
    assign C[4] = G[3] | (P[3] & G[2]) | (P[3] & P[2] & G[1]) |
                  (P[3] & P[2] & P[1] & G[0]) |
                  (P[3] & P[2] & P[1] & P[0] & C[0]);

    assign Cout = C[4];
endmodule
```

4 4

A B Cin 4 S Cout

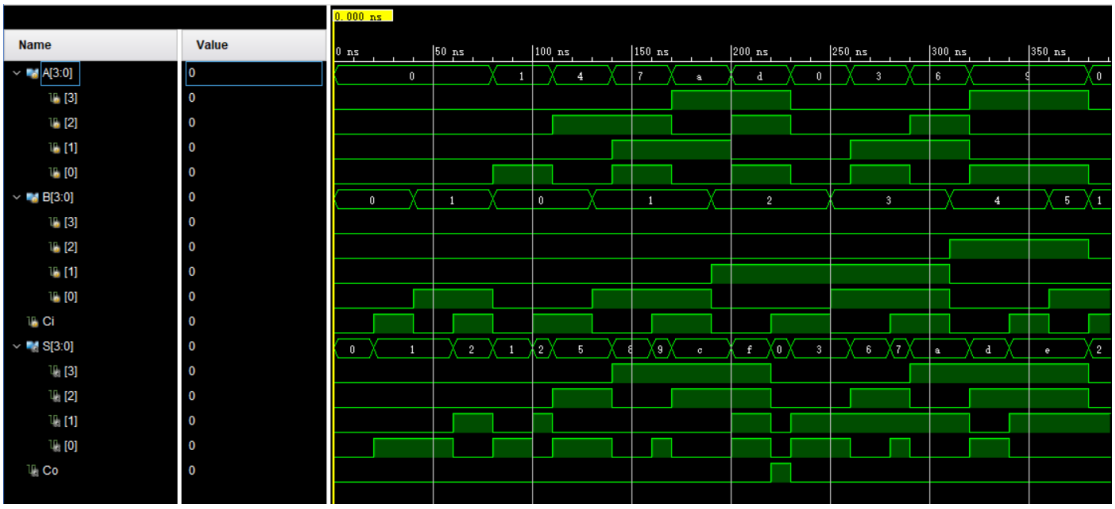
G P C

- G A[i] B[i] 1 G[i] = 1
- P A[i] B[i] 1 P[i] = 1
- C Cin

generate A B S[i] A[i] B[i] C[i] C[1] C[4] G P C[1] G[0] P[0]·C[0] C[4] Cout S[3:0]

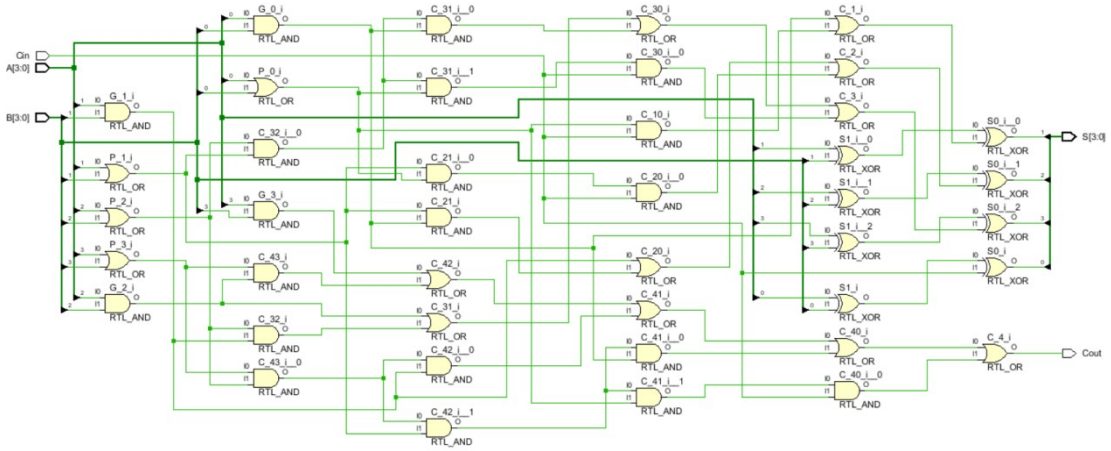
2 4

3 SIMUALTION



CLA S[3:0] Cout 4

4 RTL ANALYSIS



LU

1

1

```
module ALU(
    output reg [31:0] F,        //
    output ZF, CF, OF, SF, PF,    //
    input [31:0] A, B,          //
    input [3:0] OP                //
);

    parameter SIZE = 32;    //
```

```
//
parameter ALU_AND = 4'b0000;    //
parameter ALU_OR    = 4'b0001;    //
parameter ALU_XOR = 4'b0010;    //
parameter ALU_NOR = 4'b0011;    //
parameter ALU_ADD = 4'b0100;    //
parameter ALU_SUB = 4'b0101;    //
parameter ALU_SLT = 4'b0110;    //          A<B         1;         0
parameter ALU_SLL = 4'b0111;    //           B          A

//
wire [7:0] EN;                      //
wire [31:0] Fw, Fa;             //          F
wire CF_temp;                       //
wire C;                             //

//
assign Fa = A & B;    //

//
always@(*) begin
        case(OP)
                ALU_AND: F <= Fa;               //
                ALU_OR : F <= A | B;          //
                ALU_XOR: F <= A ^ B;          //
                ALU_NOR: F <= ~(A | B);    //
                default: F <= Fw;               //
        endcase
end

//
// 3-8                  EN
Decoder38 decoder38_1(OP[2:0], EN);

//
ADD add_1(Fw, CF_temp, A, B, EN[4]);
```

```
        //
        SUB sub_1(Fw, CF_temp, A, B, EN[5]);

        //          A<B         1;         0
        SLT slt_1(Fw, A, B, EN[6]);

        //           B          A
        SLL sll_1(Fw, A, B, EN[7]);

        assign C = (OP == ALU_ADD) ? CF_temp :
                        (OP == ALU_SUB) ? ~CF_temp : 1'b0;
        //
        assign ZF = (F == 0);                                                   //
        assign CF = (OP == ALU_ADD || OP == ALU_SUB) ? CF_temp : 1'b0;    //
        assign OF = A[31] ^ B[31] ^ F[31] ^ CF;                         //
        assign SF = F[31];                                                      //
        assign PF = ~^F;                                                        //
endmodule
```

2 3-8

```
module Decoder38(A, Y);
        input [2:0] A;
        output reg [7:0] Y;
        integer i;

        always @(A) begin
                Y = 8'b1111_1111;
                for(i = 0; i<=7; i = i + 1)
                        if (A == i)
                                Y[i] = 0;
                        else
                                Y[i] = 1;
        end
endmodule
```

3

```
module ADD(
        output reg [31:0] F,
```

```
        output reg CF,
        input [31:0] A, B,
        input EN
);
        always@(*) begin
                if(!EN) begin
                        {CF, F} = A + B;
                end else begin
                        F = 32'bz;
                        CF = 1'bz;
                end
        end
endmodule
```

4

```
module SUB(
        output reg [31:0] F,
        output reg CF,
        input [31:0] A, B,
        input EN
);
        always@(*) begin
                if(!EN) begin
                        {CF, F} = A - B;
                end else begin
                        F = 32'bz;
                        CF = 1'bz;
                end
        end
endmodule
```

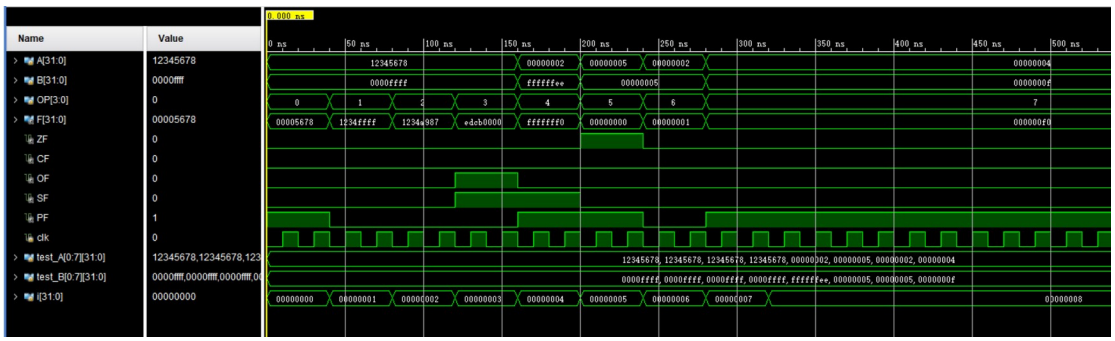
5

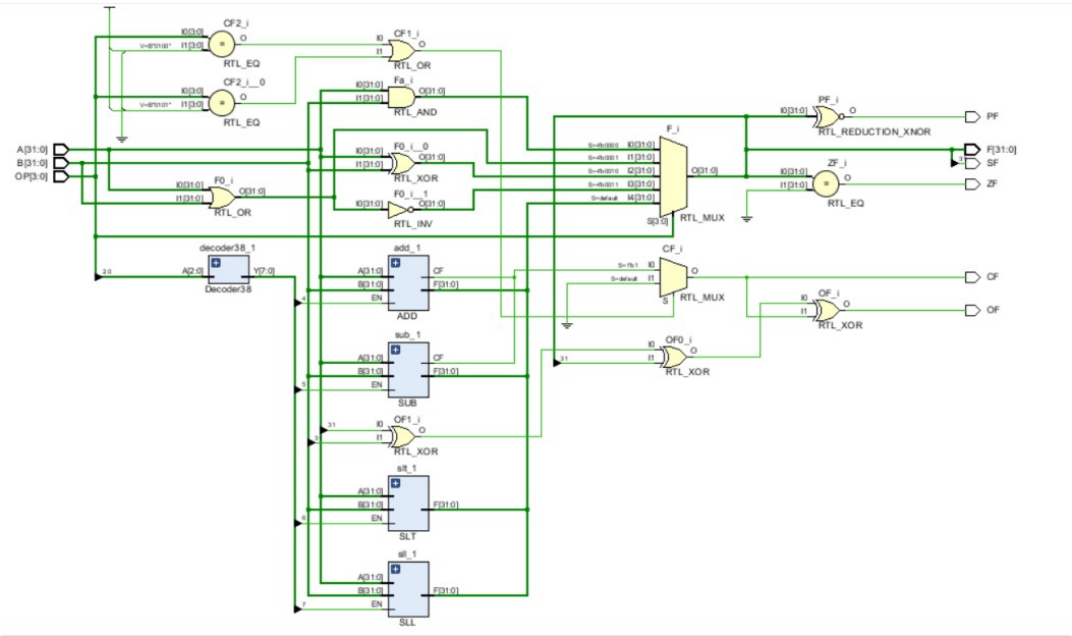
```
module SLL(
        output reg [31:0] F,
        input [31:0] A, B,
        input EN
);
        always@(*) begin
```

```
        if(!EN) begin
            F = B << A[4:0];    //          A      5            31
        end else begin
            F = 32'bz;
        end
    end
endmodule

  6
module SLT(
    output reg [31:0] F,
    input [31:0] A, B,
    input EN
);
    always@(*) begin
        if(!EN) begin
            F = (A < B) ? 32'b1 : 32'b0;
        end else begin
            F = 32'bz;
        end
    end
endmodule

  7

2
module sim_ALU;
    reg [31:0] A, B;
    reg [3:0] OP;
    wire [31:0] F;
    wire ZF, CF, OF, SF, PF;

    ALU alu(
        .F(F), .ZF(ZF), .CF(CF), .OF(OF), .SF(SF), .PF(PF),
        .A(A), .B(B), .OP(OP)
    );
    reg clk;
    reg [31:0] test_A [0:7];
```

```
    reg [31:0] test_B [0:7];
    initial begin
        //
        test_A[0] = 32'h12 345 678; test_B[0] = 32'h0000FFFF;    // AND
        test_A[1] = 32'h12 345 678; test_B[1] = 32'h0000FFFF;    // OR
        test_A[2] = 32'h12 345 678; test_B[2] = 32'h0000FFFF;    // XOR
        test_A[3] = 32'h12 345 678; test_B[3] = 32'h0000FFFF;    // NOR
        test_A[4] = 32'h000 000f2; test_B[4] = 32'hfffffffee;    // ADD
        test_A[5] = 32'h00 000 005; test_B[5] = 32'h00 000 005;    // SUB
        test_A[6] = 32'h00 000 002; test_B[6] = 32'h00 000 005;    // SLT
        test_A[7] = 32'h00 000 004; test_B[7] = 32'h0 000 000F;    // SLL
    end
    initial begin
        clk = 0;
        forever #10 clk = ~clk;
    end
    integer i;
    initial begin
        i = 0; A = 0; B = 0; OP = 0;
        for (i = 0; i < 8; i = i + 1) begin
            A = test_A[i];
            B = test_B[i];
            OP = i;
            #40;
        end
    end
endmodule
```

3 SIMULATION





4 RTL ANALYSIS:

五、调试和心得体会